大模型技术之 Hugging Face

第 1 章 Hugging Face 生态概览

1.1 简介

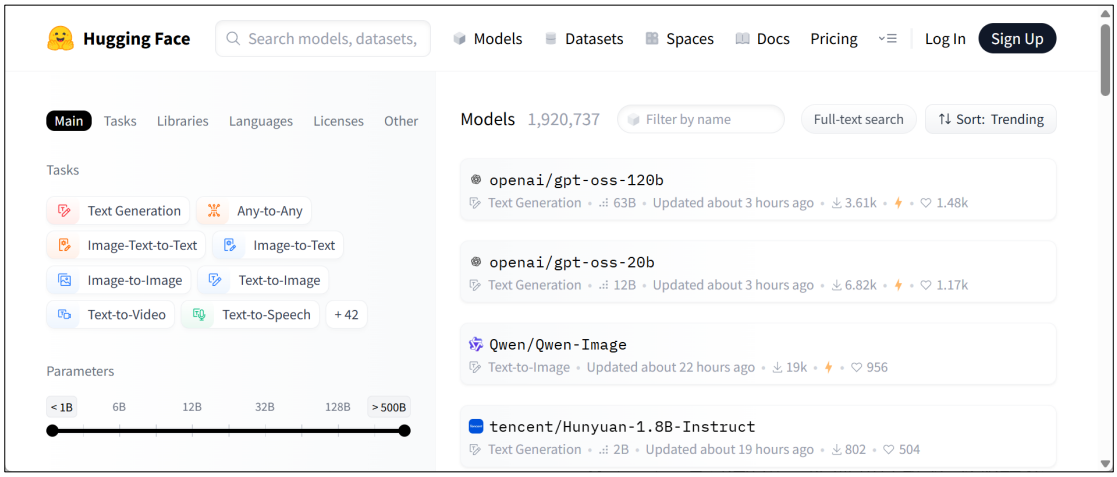
Hugging Face 是一家提供开源 AI 工具和平台的公司，致力于简化预训练模型的使用，加速机器学习项目的开发与落地。

最初以 Transformers 库闻名，该库极大地降低了使用 BERT、GPT、T5 等模型的门槛。如今，Hugging Face 已发展成为一个完整的 AI 开发生态系统，支持自然语言处理、计算机视觉、语音处理、多模态任务等多个领域。

Hugging Face 的生态系统主要由两个核心部分组成：

1）Hugging Face Hub

Hugging Face 提供了一个集中式的开源平台，用于托管和分享模型、数据集和应用。



官网地址为：https://huggingface.co/

国内镜像地址为：https://hf-mirror.com/

2）工具链（Libraries）

Hugging Face 提供了一套围绕预训练模型构建的工具库。这些组件彼此独立，又可以协同工作，覆盖了从数据处理到模型训练与推理的完整流程。

各组件具体功能如下：

- **Datasets**

Datasets 是用于加载和处理数据集的工具库。支持从在线仓库或本地文件（如 CSV、JSON）加载文本数据，并支持清洗、编码、切分等预处理操作。处理后的数据可直接用于模型训练，是连接原始数据与模型输入的重要桥梁。

➢ **Tokenizers**

Tokenizers 是用于将文本转换为模型输入的工具。它支持文本分词、编码为 token ID，同时自动处理特殊符号、填充（padding）、attention mask 和句子对标记（token type ID）。分词器通常与模型配套使用，可通过统一接口加载。

➢ **Transformers**

Transformers 是 Hugging Face 最核心的库，用于加载、使用和微调各种预训练模型。该库统一了模型接口，支持数百种模型结构，如 BERT、GPT 等，用户可以通过一行代码 from_pretrained()直接加载公开模型，快速用于推理或训练。

第 2 章 预训练模型的加载与使用

2.1 模型加载详解

2.1.1 AutoModel 类

在使用 Hugging Face 生态中的预训练模型时，第一步往往是从 Hub 上选择一个合适的模型，然后将其加载到本地进行微调或推理。为了简化这一流程，Transformers 库提供了统一的模型加载接口—— AutoModel，用于自动下载和加载模型。

具体代码如下：

```
from transformers import AutoModel

# 加载模型
model = AutoModel.from_pretrained("google-bert/bert-base-chinese")
```

上述代码执行的操作如下：

1）下载模型所需资源

AutoModel 会根据提供的模型名称，从 Hugging Face Hub 上下载所需的模型资源，包括模型权重和配置文件。

这些文件会自动缓存到本地，默认路径是：~/.cache/huggingface/hub/。下次加载相同模型时会直接读取缓存，不再联网下载。

注意：如需使用国内镜像站，需配置如下环境变量

```
HF_ENDPOINT=https://hf-mirror.com
```

2）根据配置文件创建模型

配置文件（config.json）定义了模型的结构信息，Transformers 会据此识别模型类型（如 BERT），并自动实例化对应的模型类（如 BertModel）。这些模型类均继承自 PyTorch 的 nn.Module，因此构建出的对象本质上是一个标准的神经网络模型。

上述代码得到的 model 类型为 BertModel。

3）加载模型权重

将下载的权重文件加载到模型实例中，至此模型准备完毕，可直接用于推理或微调。

除了在线加载模型之外，from_pretrained()也支持从本地路径加载模型，要求目录中包含模型权重和配置文件，代码如下

```
from transformers import AutoModel

# 加载模型
model = AutoModel.from_pretrained("./pretrained/bert-base-chinese")
```

2.1.2 AutoModelForXXX 类

AutoModel 只加载预训练模型的主干结构，不包含任何任务相关的输出层，适用于特征提取或自定义模型结构的场景。

除此之外，Transformers 还提供了用于具体任务的专用模型类：AutoModelForXXX，这些类在模型主干的基础上，自动添加了适配任务的输出层（通常称为“任务头”或 Task Head），使模型能够直接用于分类、命名实体识别、问答等标准 NLP 任务的训练与推理，无需手动修改结构。

常用的任务模型类有：

模型类名称	适用任务类型	简要说明
AutoModelForCausalLM	自回归语言建模	用于根据已有文本逐字生成后续文本
AutoModelForSeq2SeqLM	序列到序列生成任务	适用于机器翻译、文本摘要、对话生成
AutoModelForSequenceClassification	文本分类	例如情感分析、主题分类、多标签分类
AutoModelForTokenClassification	Token 级别标注任务	如命名实体识别（NER）
AutoModelForQuestionAnswering	抽取式问答	用于从上下文中找出答案的起始和结束位置

上述 AutoModelForXXX 类的用法与 AutoModel 类一致，例如现在需要一个基于 bert-base-chinese 的文本分类模型，便可直接通过以下代码进行加载：

```
# 加载模型
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained("google-bert/bert-base-chinese")
```

上述代码得到的 model 的类型为 BertForSequenceClassification。模型结构包括：

- BERT 编码器主干；
- 一个线性层（任务头），用于输出每个类别的得分。

此外，对于特定任务的模型，我们还可以在 from_pretrained() 中设置一些参数用于控制任务头的行为，例如：

```
model = AutoModelForSequenceClassification.from_pretrained(
    "google-bert/bert-base-chinese",
    num_labels=3
)
```

参数说明：

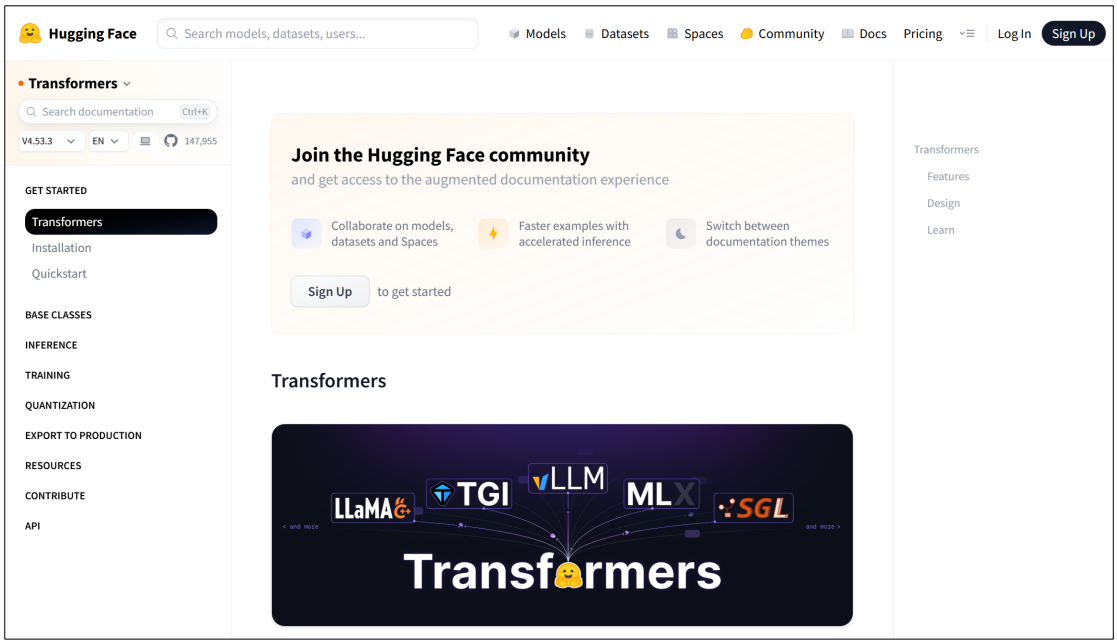
参数名	说明
num_labels	指定分类任务的类别数，默认值为 2。用于构建分类头的输出维度

2.2 模型输入输出详解

在使用 Hugging Face 的 Transformers 模型时，理解其输入格式与输出结构，是正确使用模型的前提。

由于通过 AutoModel 或 AutoModelForXXX 加载的模型，本质上是 PyTorch 的 nn.Module 子类，其前向传播过程通过 forward() 方法实现，所以要了解某个模型支持哪些输入参数、返回哪些输出字段，最直接、最权威的方式就是查看其 forward() 方法定义。

各模型 forward 方法的定义，可查看 Transformers 库的 API 文档：



例如：

BertModel 的 forward 方法定义可参考如下链接

- **官方网站：**

https://huggingface.co/docs/transformers/model_doc/bert#transformers.BertModel.forward

- **镜像网站：**

https://hf-mirror.com/docs/transformers/model_doc/bert#transformers.BertModel.forward

BertForSequenceClassification 的 forward 方法定义可参考如下链接

- **官方网站：**

https://huggingface.co/docs/transformers/model_doc/bert#transformers.BertForSequenceClassification.forward

- **镜像网站：**

https://hf-mirror.com/docs/transformers/model_doc/bert#transformers.BertForSequenceClassification.forward

第 3 章 Tokenizer 的加载与使用

3.1 概述

在 Hugging Face 的 Transformers 库中，每一个预训练模型都配套绑定有一个专用的 Tokenizer，它负责将原始文本转换为模型可以理解的输入格式（如 input_ids、attention_mask 等），是连接原始文本与模型计算之间的关键环节。

这些 Tokenizer 通常集成了从文本到张量的全流程处理能力，主要包括以下几个方面：

- 子词切分（subword tokenization）：将输入文本拆分为子词单元；
- 编码映射：将每个子词转换为对应的整数 ID，即 input_ids；
- 添加特殊 Token：自动插入如 [CLS]、[SEP] 等任务相关的特殊符号；
- 截断与补齐（truncation & padding）：统一输入序列长度，构造批量输入；
- 生成辅助输入：根据模型需求生成 attention_mask、token_type_ids 等附加字段；

3.2 加载 Tokenizer

在 Transformers 库中，AutoTokenizer 用于加载与指定模型配套的分词器。它会根据模型名称自动选择并实例化正确的分词器类型（如 BertTokenizer、GPT2Tokenizer、T5Tokenizer 等）。

AutoTokenizer 的用法与 AutoModel 相似，具体用法如下：

```
from transformers import AutoTokenizer

# 加载分词
tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-chinese")
```

上述代码执行的操作如下：

AutoTokenizer 会根据提供的模型名称，从 Hugging Face Hub 上下载所需的文件资源，包括配置文件词表。这些文件会自动缓存到本地，默认路径是：~/.cache/huggingface/hub/。下次加载相同模型时会直接读取缓存，不再联网下载。

注意：如需使用国内镜像站，需配置如下环境变量

```
HF_ENDPOINT=https://hf-mirror.com
```

之后 AutoTokenizer 便会根据配置文件和词表实例化一个 Tokenizer 对象。

除了在线加载模型之外，from_pretrained()也支持从本地路径加载模型，要求目录中包含词表和配置文件，代码如下

```
from transformers import AutoTokenizer

# 加载模型
tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")
```

3.3 使用 Tokenizer

3.3.1 概述

前文提到过，Transformers 库中的 Tokenizer 包括如下功能：

- 子词切分
- 编码映射
- 添加特殊 Toke
- 截断与补齐
- 生成辅助输入

下面逐一进行演示：

3.3.2 常用 API

1）分词（tokenize）

```
from transformers import AutoTokenizer

# 加载分词器和模型
tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")

tokens = tokenizer.tokenize("我爱自然语言处理")

print(tokens)
```

输出内容如下

```
['我', '爱', '自', '然', '语', '言', '处', '理']
```

2）token 转 ID（convert_tokens_to_ids）

```
from transformers import AutoTokenizer

# 加载分词器和模型
tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")

tokens = tokenizer.tokenize("我爱自然语言处理")

ids = tokenizer.convert_tokens_to_ids(tokens)

print(ids)
```

输出内容如下

```
[2769, 4263, 5632, 4197, 6427, 6241, 1905, 4415]
```

3）ID 转 token（convert_ids_to_tokens）

```
from transformers import AutoTokenizer

# 加载分词器和模型
tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")

ids = [2769, 4263, 5632, 4197, 6427, 6241, 1905, 4415]

tokens = tokenizer.convert_ids_to_tokens(ids)
```

```
print(tokens)
```

输出内容如下

```
['我', '爱', '自', '然', '语', '言', '处', '理']
```

4）编码（encode）

编码是将 tokenize + convert_tokens_to_ids 合并后的结果，通常还会自动添加特殊符号（如 [CLS] 和 [SEP]），除此之外，还支持 padding、truncate 等功能。

```
from transformers import AutoTokenizer

# 加载分词器和模型
tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")

ids = tokenizer.encode("我爱自然语言处理")

print(ids)
```

输出内容如下

```
[101, 2769, 4263, 5632, 4197, 6427, 6241, 1905, 4415, 102]
```

注：可通过 add_special_tokens=False 参数禁止添加特殊符号

5）解码（decode）

解码会将一个 token ID 序列还原为对应的原始文本（或接近的文本）。

```
from transformers import AutoTokenizer

# 加载分词器和模型
tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")

ids = [101, 2769, 4263, 5632, 4197, 6427, 6241, 1905, 4415, 102]

string = tokenizer.decode(ids)

print(string)
```

输出内容如下：

```
[CLS] 我 爱 自 然 语 言 处 理 [SEP]
```

注：可通过 skip_special_tokens=True 参数跳过特殊符号

6）tokenizer() 方法（即 __call__）

这是最推荐的接口，用于直接构造模型所需的输入，其基本用法如下

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")
text = "我爱自然语言处理"

# 编码文本为模型输入格式
inputs = tokenizer(text)

print(inputs)
```

输出内容如下：

```
{
  'input_ids': [101, 2769, 4263, 5632, 4197, 6427, 6241, 1905, 4415, 102],
  'token_type_ids': [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
}
```

除去 text，tokenizer 还提供了多个重要参数：

```python
inputs = tokenizer(
    text,
    padding=True,
    truncation=True,
    max_length=128,
    return_tensors="pt"
)
```

各参数含义如下请参考[官方文档](官方文档)。

此外，tokenizer()方法还支持直接对多个文本组成的列表进行**批量**处理，非常适合用于模型训练或推理。

```python
from transformers import AutoTokenizer

# 加载分词器和模型
tokenizer = AutoTokenizer.from_pretrained("./pretrained/bert-base-chinese")

texts = ["我爱自然语言处理", "我爱人工智能", "我们一起学习"]
inputs = tokenizer(
    texts,
    padding="max_length",  # 自动补齐
    truncation=True,  # 自动截断
    max_length=10,  # 统一最大长度
    return_tensors="pt"  # 返回 PyTorch 张量格式
)

print(inputs)
```

输出内容是一个包含三个字段的字典，每个字段是形状为 (batch_size, seq_len) 的张量：

```
{
    'input_ids': tensor([[ 101, 2769, 4263, 5632, 4197, 6427, 6241, 1905, 4415,  102],
                         [ 101, 2769, 4263,  782, 2339, 3255, 5543,  102,    0,    0],
                         [ 101, 2769,  812,  671, 6629, 2110,  739,  102,    0,    0]]),
    'token_type_ids': tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                              [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
                              [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]),
    'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                              [1, 1, 1, 1, 1, 1, 1, 1, 0, 0],
                              [1, 1, 1, 1, 1, 1, 1, 1, 0, 0]])
}
```

3.4 与预训练模型配合使用

从文本输入到模型输出的完整流程如下：

```python
from transformers import AutoTokenizer, AutoModel
import torch

# 1. 加载模型和分词器
```

```python
model_name = "bert-base-chinese"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

# 2. 准备批量文本
texts = ["我爱自然语言处理", "我爱人工智能", "我们一起学习"]

# 3. 编码文本为模型输入格式
encoded = tokenizer(
    texts,
    padding="max_length",
    truncation=True,
    max_length=10,
    return_tensors="pt"
)

# 5. 模型推理（不计算梯度）
with torch.no_grad():
    outputs = model(
        input_ids=encoded["input_ids"],
        attention_mask=encoded["attention_mask"],
        token_type_ids=encoded["token_type_ids"]
    )

# 6. 查看输出张量结构
print(outputs.keys())
print("last_hidden_state:", outputs.last_hidden_state.shape)
print("pooler_output:", outputs.pooler_output.shape)
```

输出内容如下：

```
odict_keys(['last_hidden_state', 'pooler_output'])
last_hidden_state: torch.Size([3, 10, 768])
pooler_output: torch.Size([3, 768])
```

第 4 章 Datasets 库

4.1 概述

datasets 是 Hugging Face 提供的一个轻量级数据处理库，专为自然语言处理任务设计，能够高效地支持模型训练流程中的数据加载与预处理操作。

它的主要特点包括：

- 加载方便：支持读取本地文件（如 CSV、JSON），也支持加载在线公开数据集；
- 结构清晰：数据集的内部结构类似表格，每条样本由若干字段组成；
- 无缝协作：与 tokenizer 等 Hugging Face 模块高度集成，可直接构造模型输入；
- 功能丰富：支持常见的数据处理操作，如批量映射（.map()）、字段筛选、训练/验证集划分（.train_test_split()）等。

datasets 库的安装命令如下：

```
pip install datasets
```

4.2 加载数据集

datasets 库提供了统一的接口 load_dataset()，既支持从本地文件加载数据，也支持从 Hugging Face Hub 加载在线开源数据集。

4.2.1 加载本地数据

load_dataset()支持多种本地文件格式，如 CSV、JSON、Parquet，并允许一次加载一个或多个文件。其基本语法如下：

```
from datasets import load_dataset

dataset = load_dataset(format, data_files=路径或字典)
```

参数说明如下：

参数	类型	说明
format	str	文件格式，常用的包括 "csv"、"json"、"parquet" 等
data_files	str 或 dict	文件路径。可传入字符串（加载单个文件）或字典（加载多个文件，如训练数据/测试数据）

具体用法如下：

1）加载多个文件

```
from datasets import load_dataset

dataset_dict = load_dataset('csv', data_files={
    'train': './data/train.csv',
    'test': './data/test.csv'
})
```

此时返回的是一个包含两个 Dataset 的 DatasetDict，其中每个 Dataset 称为一个 split。

```
from datasets import load_dataset

dataset_dict = load_dataset('csv', data_files={
    'train': './data/train.csv',
    'test': './data/test.csv'
})


print(dataset_dict)
# DatasetDict({
#     train: Dataset(...),
#     test: Dataset(...)
# })
```

2）加载单个文件

```
from datasets import load_dataset

dataset_dict = load_dataset('csv', data_files='./data/dataset.csv')
```

此时返回的也是一个 DatasetDict，其中只包含默认命名为 "train" 的一个 Dataset。

```
print(dataset_dict)
# DatasetDict({
#     train: Dataset(...)
# })
```

4.2.2 查看数据集

本节以情感分析案例中的评论数据集为例，演示如何使用 `datasets` 的常用 API 查看数据内容：

1）获取 Dataset

load_dataset()返回的是一个 DatasetDict 对象，可以像字典一样通过键名（如 "train"）访问 split。

```
from datasets import load_dataset

dataset_dict = load_dataset('csv', data_files='data/raw/online_shopping_10_cats.csv')

dataset = dataset_dict["train"]
```

此时 dataset 是一个 `Dataset` 对象，表示训练集。

2）访问样本

Dataset 支持索引和切片操作来访问样本：

```
print(dataset[0])       # 单条样本
print(dataset[:3])      # 多条样本（注意返回结构）
```

返回结构说明：

访问方式	返回示例
dataset[0]	{'review': '很喜欢的一本书', 'label': 1, 'cat': '书籍'}
dataset[:3]	{'review': ['很喜欢的一本书', '内容丰富', '讲解清晰'], 'label': [1, 1, 1], 'cat': ['书籍','书籍','书籍']}

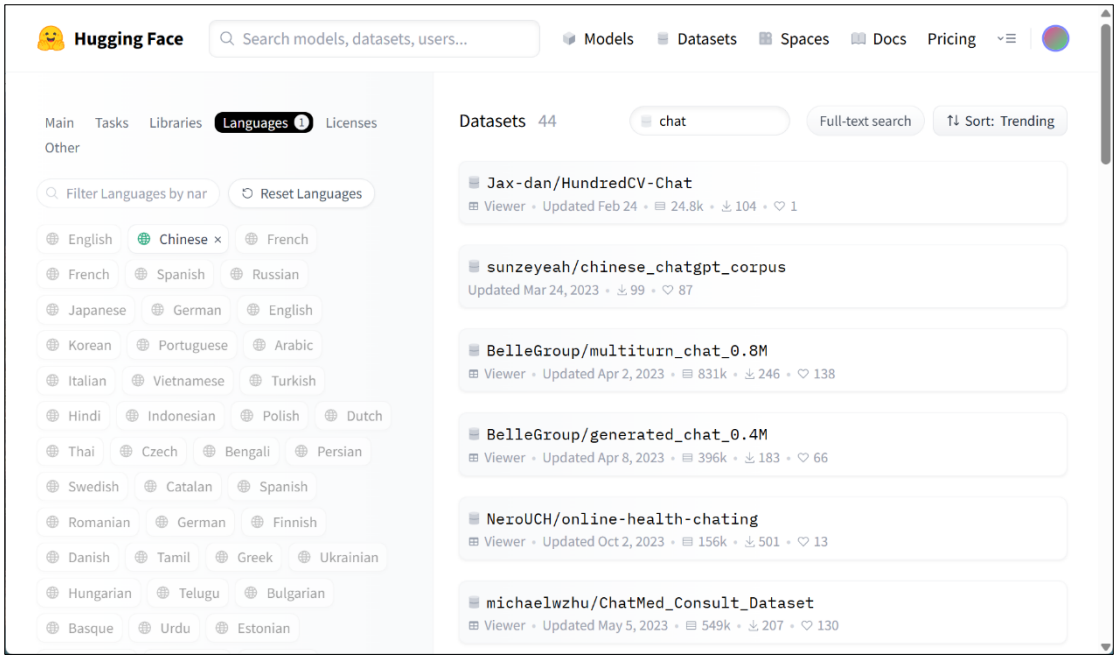
3）访问某个字段值

可以进一步通过字段名访问某个字段的值：

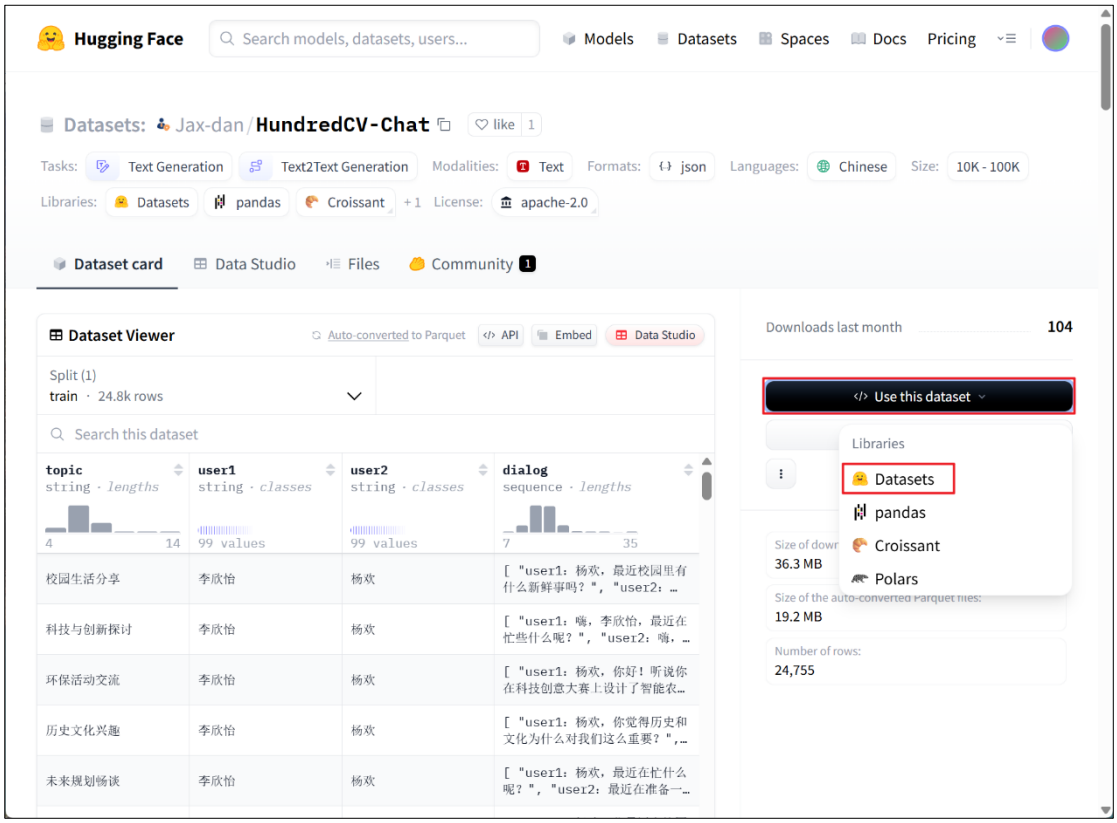
```
print(dataset[0]['review'])        # 第一条样本的 review 字段
print(dataset[:3]['review'])       # 前三条样本的 review 字段列表
```

4.2.3 加载在线数据

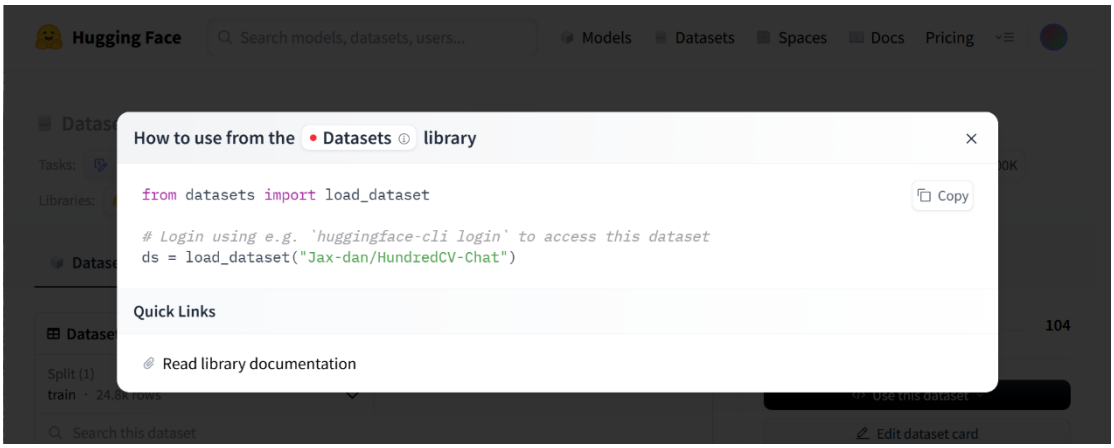
Hugging Face Hub 提供了大量开源数据集，涵盖文本分类、问答、翻译、摘要等任务，可以在官网浏览与搜索：



每个数据集页面都会提供示例代码，方便直接复制使用。



具体代码如下图所示：



执行上述代码时，数据集会自动从 Hugging Face Hub 下载，并缓存至本地用户目录，默认路径为：~/.cache/huggingface/datasets/

后续再次使用时将自动从本地加载，无需联网或重复下载。

加载完成后，返回一个 DatasetDict 对象，结构和使用方式与本地数据完全一致。

4.3 预处理数据集

除了加载数据， datasets 库还支持常见的数据预处理操作，如编码文本、删除列、过滤样本、划分子集和设置张量格式。本节将逐步介绍这些功能。

4.3.1 删除列

可通过 .remove_columns() 删除不再需要的字段

```
dataset = dataset.remove_columns(["cat"])
```

4.3.2 过滤行

可使用 .filter() 筛选符合条件的样本

```
dataset = dataset.filter(lambda x: x["review"] is not None and x["review"].strip() != ""
and x["label"] in [0, 1])
```

4.3.3 划分数据集

可使用 .train_test_split() 将单一数据集划分为训练集和验证集：

```
dataset_dict = dataset.train_test_split(test_size=0.2)

train_dataset = dataset_dict["train"]
test_dataset = dataset_dict["test"]
```

4.3.4 编码数据

可使用.map()方法与 tokenizer 配合，将原始文本批量编码为模型可用的输入格式（如 input_ids、attention_mask、token_type_ids 等）。

.map()是 datasets 中的核心方法之一，支持对整个数据集中的每一条样本或每一批样本进行统一处理，常用于文本编码（tokenizer）和数据字段换。.map() 方法基本语法如下：

```
dataset = dataset.map(function, batched=False, remove_columns=None)
```

参数说明如下：

参数	说明
function	要应用到每条样本上的函数（或每批样本上的函数）
batched	是否以“批”为单位处理样本；若为 True，则每次接收一个样本列表
remove_columns	是否删除原始列，常用于清理不再需要的字段

以中文 BERT 模型为例，编码流程如下：

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-chinese")

def tokenize(example):
    encoded =  tokenizer(
        example["review"],
        padding="max_length",
        truncation=True,
        max_length=128
    )
    example['input_ids'] = encoded['input_ids']
    example['attention_mask'] = encoded['attention_mask']

    return example

train_dataset = train_dataset.map(tokenize, batched=True)
test_dataset = test_dataset.map(tokenize, batched=True)
```

编码后，数据集中将新增字段如 input_ids 和 attention_mask，可直接用于模型训练。

4.4 保存数据集

处理后的数据可保存到本地，供后续训练或复用，避免重复预处理。 Datasets 提供了多种保存方式，适用于不同场景：

数据格式	保存方法	适用对象
Arrow	save_to_disk()	Dataset 或 DatasetDict
CSV	to_csv()	仅限 Dataset
JSON	to_json()	仅限 Dataset

4.4.1 Arrow 格式

Arrow 格式是 Hugging Face 官方推荐的数据持久化方式，既支持单个 Dataset 也支持多个子集的 DatasetDict。

- **保存**

```
dataset_dict.save_to_disk("./data/processed")
```

保存后的目录结构示例：

```
processed/
├── dataset_dict.json
├── test/
│    ├── data-00000-of-00001.arrow
│    ├── dataset_info.json
│    └── state.json
└── train/
    ├── data-00000-of-00001.arrow
    ├── dataset_info.json
```

```
    └── state.json
```

每个 split（如 train、test）都会单独保存一个 Arrow 文件和相应的元数据。

- **加载**

```
from datasets import load_from_disk

dataset_dict = load_from_disk("./data/processed")
```

4.4.2 CSV 和 JSON 格式

如果希望将数据导出为通用格式（如用于可视化或非 Hugging Face 工具使用），可以使用 .to_csv() 或 .to_json() 方法。但需注意，这些方法仅适用于单个 Dataset，不支持 DatasetDict。

- **保存**

```
# csv
train_dataset.to_csv("./data/processed/train.csv")

# json
train_dataset.to_json("./data/processed/train.json")
```

- **加载**

使用 load_dataset()，指定格式和路径即可重新加载：

```
from datasets import load_dataset

# 加载 CSV 文件
dataset_dict = load_dataset("csv", data_files="./data/processed/train.csv")

# 加载 JSON 文件
dataset_dict = load_dataset("json", data_files="./data/processed/train.json")
```

加载后返回一个结构完整的 DatasetDict，可直接用于训练、评估等任务。

4.5 集成 Dataloader

经过预处理的 datasets.Dataset 对象可以直接与 PyTorch 的 DataLoader 集成使用。虽然它并非继承自 torch.utils.data.Dataset 类，但由于实现了__len__()和__getitem__()这两个核心接口，因此能够被 DataLoader 正确识别并进行批量迭代。

在使用前，需要通过.set_format()方法将指定字段转换为张量格式以适配模型输入。典型配置如下：

```
train_dataset.set_format(
    type="torch",  # 指定输出为 PyTorch 张量
    columns=["input_ids", "attention_mask", "label"]  # 需要转换的字段
)
```

需要注意的是：

- 该方法仅改变通过__getitem__()（即 dataset[i]）访问样本时的返回格式，不会修改底层数据存储
- 通过 columns 指定的字段会在访问时自动转换为 torch.Tensor 类型
- 未通过 columns 指定的字段在访问时将被自动过滤

完成格式设置后，即可创建标准的 DataLoader 实例：

```python
from torch.utils.data import DataLoader

# 训练集 DataLoader
train_dataloader = DataLoader(train_dataset, batch_size=32, shuffle=True)
```